Plan de pruebas: Seleccionar la ciudad

Aplicación Bicis · Grupo 3 · Historia de usuario 532301 · Autor: Diego Alonso

Historia de usuario

Seleccionar la ciudad (ID 532301). Yo, como visitante temporal, quiero seleccionar la ciudad en la que voy a usar las bicis de manera que vea directamente sus estaciones sin tener que buscar entre todas.

Interfaz esperada: mock-up de la historia (Docs/Mockups/US532301-SeleccionarCiudad-Mockups.pdf). En la barra superior de la lista principal de redes hay un nuevo icono con forma de chincheta. Al pulsarlo aparecen dos desplegables, «País» y «Ciudad», encima de la lista de redes. El desplegable de países tiene como primera opción «Todos»; la lista de redes se actualiza cada vez que se cambia el país o la ciudad.

Niveles de prueba

Los niveles de prueba que se van a aplicar son los siguientes:

- **Pruebas de aceptación.** Son los criterios de aceptación acordados con el Product Owner. Se ejecutan de forma manual en el Sprint Review sobre la versión de depuración, con los datos del Anexo 1, porque las pruebas necesitan una ciudad con dos redes y redes en varios países, y la API real no lo garantiza.
- **Pruebas de interfaz.** Un caso de éxito y uno de error, implementados con Espresso sobre MainView con los datos del Anexo 1. La ubicación y el repositorio se sustituyen con Hilt cuando la prueba lo necesita.
- **Pruebas unitarias.** Se aplican a los métodos nuevos de CitySelector (modelo) y de MainPresenter, con técnicas de caja negra (partición equivalente y análisis de valores límite), usando JUnit. En el presenter la vista, el repositorio y el proveedor de ubicación se sustituyen por mocks de Mockito.
- **Pruebas de integración.** Se prueba MainPresenter con el modelo (CitySelector) y el repositorio reales (JsonNetworksRepository con los datos del Anexo 1) y solo la vista y la ubicación simuladas con Mockito.

A continuación, se muestra una especificación detallada de los casos de prueba a aplicar en cada nivel.

Criterios de aceptación

Todas las pruebas parten de la lista principal de redes, que es la primera pantalla de la aplicación. La selección se abre con el icono de la chincheta de la barra superior. Los datos concretos de cada prueba son los del Anexo 1 (Tabla 9): 13 redes en cuatro países, con dos redes en Madrid (BiciMAD y Bici Metro).

En las pruebas 1, 2, 3, 5 y 6 la ubicación del usuario no está disponible, para que la aplicación se quede en la lista de redes y no abra las estaciones de la ciudad detectada (historia 532319, Detectar la ciudad).

Prueba 1. Caso de éxito

Contexto: hay varias redes en distintos países y ciudades, y al menos una ciudad tiene más de una red (Madrid, con BiciMAD y Bici Metro).

1. El usuario pulsa el icono de la chincheta en la lista principal de redes.
2. La aplicación muestra dos desplegables: país y ciudad.

3. El usuario selecciona «España» en el primer desplegable.
4. El usuario selecciona «Madrid» en el segundo desplegable.
5. Se verifica que la lista muestra todas las redes de la ciudad seleccionada: BiciMAD y Bici Metro (2 redes).
6. Se verifica que no aparece ninguna red de otra ciudad (por ejemplo, TUeBICI, de Santander).
7. El usuario selecciona BiciMAD y se verifica que se abre la lista de estaciones de esa red.
8. El usuario vuelve atrás y se verifica que la lista sigue mostrando solo las redes de Madrid.

Prueba 2. Orden alfabético de países y ciudades

Contexto: hay redes en varios países (Alemania, Argentina, España y Portugal), y algún país tiene redes en varias ciudades (España, en nueve).

1. El usuario pulsa el icono de la chincheta en la lista principal de redes.
2. El usuario abre el desplegable de países.
3. Se verifica que los países aparecen con su nombre en español y ordenados alfabéticamente: «Todos», «Alemania», «Argentina», «España», «Portugal».
4. El usuario selecciona «España» y abre el desplegable de ciudades.
5. Se verifica que las ciudades aparecen ordenadas alfabéticamente: Barcelona, Bilbao, Burgos, Cádiz, Gijón, León, Madrid, Oviedo, Santander.

Nota: la API solo da el código del país (por ejemplo, «DE»). El nombre en español se obtiene a partir de ese código, y se ordena por el nombre, no por el código: por eso «Alemania» (DE) va antes que «Argentina» (AR). Las ciudades se ordenan con las reglas del español, de modo que una ciudad con tilde (por ejemplo, «Ávila») va en su sitio y no al final.

Prueba 3. Solo aparecen ciudades con red

Contexto: el país que se va a elegir tiene varias ciudades con redes de bicicletas (España).

1. El usuario pulsa el icono de la chincheta en la lista principal de redes.
2. El usuario selecciona «España» en el primer desplegable.
3. Se verifica que el desplegable de ciudades solo contiene ciudades de ese país que tienen al menos una red de bicicletas (las nueve de la Prueba 2).
4. Se verifica que no aparece ninguna ciudad repetida, aunque tenga varias redes («Madrid» aparece una sola vez).
5. El usuario cambia el país seleccionado a «Portugal».
6. Se verifica que el desplegable de ciudades se actualiza y solo contiene ciudades del nuevo país («Lisboa»).
7. Se verifica que la ciudad anterior ya no está seleccionada y que la lista muestra las redes del nuevo país (Bici Lisboa).

Nota: mientras hay un país seleccionado pero ninguna ciudad, la lista muestra todas las redes de ese país. Las redes que no informan de su país o de su ciudad no aparecen en los desplegables.

Prueba 4. Ubicación actual por defecto

Contexto: la ubicación del usuario está disponible y está en una ciudad con red (Santander).

1. El usuario abre la aplicación. Se detecta Santander y se abre la lista de estaciones de TUeBICI (historia 532319).
2. El usuario vuelve atrás a la lista principal de redes y pulsa el icono de la chincheta.
3. Se verifica que en los desplegables aparecen ya seleccionados el país y la ciudad de la ubicación actual del usuario («España» / «Santander»).
4. Se verifica que la lista muestra las redes de esa ciudad (TUeBICI).

Nota: si no se puede obtener la ubicación (permiso denegado, ubicación desactivada o ninguna red a 20 km o menos), los desplegables aparecen sin selección, la lista no cambia y se muestra el mensaje «Elige el país y la ciudad en los que vas a usar las bicis».

Prueba 5. Error al cargar las redes

Contexto: no se pueden cargar las redes de bicicletas (por ejemplo, sin conexión).

1. El usuario abre la aplicación. Aparece el mensaje de error al cargar las redes de la pantalla principal.
2. El usuario pulsa el icono de la chincheta en la lista principal de redes.
3. Se verifica que aparece el mensaje «No se han podido cargar las ciudades».
4. Se verifica que los desplegables de país y ciudad no muestran ninguna opción.

Prueba 6. Quitar la ciudad seleccionada

Contexto: la lista está filtrada por una ciudad («España» / «Madrid», 2 redes).

1. El usuario pulsa el icono de la chincheta en la lista principal de redes.
2. El usuario selecciona la opción «Todos» en el desplegable de países.
3. Se verifica que la lista vuelve a mostrar todas las redes (13).
4. Se verifica que el desplegable de ciudades queda vacío y sin selección.

Pruebas de interfaz

Se implementan con Espresso en el directorio androidTest, en el paquete es.unican.bicis.activities.main, que es el de MainView, la vista que lanza los escenarios. Cada clase se nombra con el criterio de aceptación que implementa (<CriterioAceptación>UITest). Usan los datos del Anexo 1. Para que la aplicación no abra la ciudad detectada se sustituye LocationModule por un proveedor de ubicación falso que falla, como en DetectarCiudadSinUbicacionUITest. En el caso de error se sustituye además RepositoriesModule por un repositorio falso que siempre falla. Hay un caso de éxito y uno de error.

Tabla 1. Pruebas de interfaz

ID	Criterio	Implementación y entrada	Resultado esperado
UI.1	Prueba 1. Caso de éxito	SeleccionarCiudadExitoUITest. País «España», ciudad «Madrid». Después se pulsa BiciMAD y se vuelve atrás	La lista tiene 2 redes, BiciMAD y Bici Metro; se abren las estaciones de BiciMAD y al volver siguen las 2 redes
UI.2	Prueba 5. Error al cargar las redes	SeleccionarCiudadErrorCargaUITest. Repositorio falso que falla al cargar las redes	Mensaje «No se han podido cargar las ciudades» y los dos desplegables sin opciones

Las pruebas 2, 3 y 6 quedan cubiertas por las pruebas unitarias del modelo (Tablas 3 y 4) y por las de integración (Tabla 7), que usan los mismos datos. La Prueba 4 la cubren las pruebas unitarias del presenter (UMP.2b) y la integración (IMS.1f). Todas se ejecutan además a mano como pruebas de aceptación.

Métodos a probar

Métodos nuevos o modificados para esta historia de usuario. Los nombres del presenter y de la vista son los del contrato de la historia (IMainContract, tarea 978606).

Tabla 2. Métodos a probar

Método	Qué hace	Pruebas
CitySelector (modelo) getCountries(networks)	Devuelve los códigos de los países que tienen al menos una red con país y ciudad, sin repetidos y ordenados por su nombre en español. Precondición: la lista no es null.	Unitarias (Tabla 3), integración (Tabla 7)
CitySelector (modelo) getCountryName(countryCode)	Devuelve el nombre en español del país a partir de su código ISO (por ejemplo, «ES» → «España»).	Unitarias (CitySelectorTest), interfaz
CitySelector (modelo) getCities(networks, countryCode)	Devuelve las ciudades del país que tienen al menos una red, sin repetidos y en orden alfabético español. Con el país null («Todos») devuelve una lista vacía. Precondición: la lista no es null.	Unitarias (Tabla 4), integración (Tabla 7) e interfaz (UI.1)
CitySelector (modelo) filterByCity(networks, countryCode, city)	Devuelve las redes del país y la ciudad, en el mismo orden. Con la ciudad null devuelve todas las del país, y con el país null, todas las redes. Precondición: la lista no es null.	Unitarias (Tabla 5), integración (Tabla 7) e interfaz (UI.1)
MainPresenter onSelectCityClicked()	Se llama al pulsar la chincheta. Si las redes no se cargaron, pide a la vista el mensaje de error. Si no, le pide que muestre los países y, si se detectó la ciudad del usuario, la deja seleccionada y filtra la lista.	Unitarias (Tabla 6), integración (Tabla 7) e interfaz (UI.1, UI.2)
MainPresenter onCountrySelected(countryCode) onCitySelected(city)	Se llaman al cambiar el país (null = «Todos») o la ciudad. Actualizan el desplegable de ciudades y piden a la vista que muestre las redes filtradas.	Unitarias (Tabla 6), integración (Tabla 7) e interfaz (UI.1)
MainPresenter onLocationPermissionResult(granted)	Modificado: además de abrir las estaciones de la ciudad detectada, guarda la red detectada para preseleccionar su país y su ciudad (Prueba 4).	Unitarias (UMP.2b), integración (IMS.1f)
MainView showCitySelector(countries, country, city) showCities(cities, city) showCitiesLoadError() showChooseCityHint()	Muestran los desplegables con sus opciones y su selección, el mensaje de error de carga y el mensaje para que el usuario elija la ciudad.	Interfaz (UI.1, UI.2)

Pruebas unitarias

Pruebas unitarias de la clase CitySelector

Se aplican técnicas de clases de equivalencia y AVL para la definición de los casos de prueba. Además, se deberá garantizar la cobertura de sentencias y de condición/decisión. Salvo que se indique otra cosa, la lista de partida, que llamaremos L, contiene estas redes, en este orden: BiciMAD (Madrid, ES), Bici Metro (Madrid, ES), Bicing (Barcelona, ES), Bici Ávila (Ávila, ES), Bici Lisboa (Lisboa, PT), Bici Berlín (Berlín, DE), Bici Límites (Ushuaia, AR), una red sin location, una red de ES con la ciudad null, una red de Toledo con el país null y una red de ES con la ciudad en blanco (" "). Implementadas en CitySelectorTest (directorio test, paquete es.unican.bicis.model). No se prueban listas null, porque incumplen las precondiciones: MainPresenter solo llama al modelo con las redes ya cargadas. Las redes sin país o sin ciudad sí se prueban, porque son datos reales de la API.

Método getCountries(networks)

Tabla 3. Casos de prueba del método getCountries

Identificador	Entrada	Valor esperado
UCS.1a	L (Varios países con varias redes)	[DE, AR, ES, PT] (Alemania, Argentina, España, Portugal)
UCS.1b	[BiciMAD, Bici Metro] (Dos redes del mismo país)	[ES] (sin repetidos)
UCS.1c	[sin location, ciudad null, país null, ciudad en blanco] (Solo redes sin datos válidos)	[]
UCS.1d	[] (Lista vacía)	[]
UCS.1e	[BiciMAD, null] (Red null)	[ES]

Método getCities(networks, countryCode)

Tabla 4. Casos de prueba del método getCities

Identificador	Entrada	Valor esperado
UCS.2a	L, "ES" (País con varias ciudades, una con tilde y una con dos redes)	[Ávila, Barcelona, Madrid] (orden español, Madrid una vez, sin la ciudad null ni la vacía)
UCS.2b	L, "PT" (País con una ciudad)	[Lisboa]
UCS.2c	L, "FR" (País sin redes)	[]
UCS.2d	L, null («Todos»)	[]
UCS.2e	[], "ES" (Lista vacía)	[]

Método filterByCity(networks, countryCode, city)

Tabla 5. Casos de prueba del método filterByCity

Identificador	Entrada	Valor esperado
UCS.3a	L, "ES", "Madrid" (Ciudad con dos redes)	[BiciMAD, Bici Metro] (mismo orden)
UCS.3b	L, "PT", "Lisboa" (Ciudad con una red)	[Bici Lisboa]
UCS.3c	L, "ES", null (Solo país)	[BiciMAD, Bici Metro, Bicing, Bici Ávila, red con ciudad null, red con ciudad en blanco]
UCS.3d	L, "PT", "Madrid" (Ciudad de otro país)	[]

Identificador	Entrada	Valor esperado
UCS.3e	L, null, null («Todos»)	L completa, en el mismo orden
UCS.3f	[], "ES", "Madrid" (Lista vacía)	[]

Pruebas unitarias del presenter MainPresenter

Se prueba el presenter aislado. La vista (IMainContract.View), el repositorio (INetworksRepository) y el proveedor de ubicación (ILocationProvider) se sustituyen por mocks de Mockito; el repositorio responde con las redes BiciMAD (Madrid, ES), Bici Metro (Madrid, ES), TUeBICI (Santander, ES) y Bici Lisboa (Lisboa, PT). Salvo que se indique otra cosa, la ubicación falla, así que no hay ciudad detectada. Se comprueba qué métodos de la vista llama el presenter en cada caso. Implementadas en MainPresenterTest (directorio test, paquete es.unican.bicis.activities.main).

Métodos onSelectCityClicked, onCountrySelected y onCitySelected

Tabla 6. Casos de prueba del presenter

Identificador	Entrada	Valor esperado
UMP.2a	onSelectCityClicked() (Sin ciudad detectada)	showCitySelector([ES, PT], null, null) y showChooseCityHint(); no cambia la lista
UMP.2b	Ubicación en Santander (43.4623, -3.8099), onSelectCityClicked() (Ciudad detectada)	showCitySelector([ES, PT], "ES", "Santander") y showNetworks([TUeBICI])
UMP.2c	onCountrySelected("ES")	showCities([Madrid, Santander], null) y showNetworks([BiciMAD, Bici Metro, TUeBICI])
UMP.2d	onCountrySelected("ES"), onCitySelected("Madrid")	showNetworks([BiciMAD, Bici Metro])
UMP.2e	onCountrySelected("ES"), onCitySelected("Madrid"), onCountrySelected("PT") (Cambio de país)	showCities([Lisboa], null) y showNetworks([Bici Lisboa])
UMP.2f	onCountrySelected("ES"), onCitySelected("Madrid"), onCountrySelected(null) («Todos»)	showCities([], null) y showNetworks con las 4 redes
UMP.2g	Falla la carga de redes, onSelectCityClicked()	showCitiesLoadError(); no se llama a showCitySelector

Pruebas de integración

Se prueba MainPresenter con el modelo real (CitySelector, Network, Location) y el repositorio real (JsonNetworksRepository, con JUnit y Robolectric), y solo la vista y el proveedor de ubicación simulados con Mockito. Antes de cada caso se llama a init, que carga las 13 redes de res/raw/networks.json (Anexo 1), y se verifica con Mockito lo que el presenter pide a la vista. Salvo en IMS.1f, la ubicación falla. Implementadas en MainPresenterITest (directorio test, paquete es.unican.bicis.activities.main), que toma el nombre del presenter por ser la clase principal de la prueba.

Tabla 7. Casos de prueba de integración de la selección de ciudad

Identificador	Entrada	Valor esperado
IMS.1a	onSelectCityClicked()	showCitySelector([DE, AR, ES, PT], null, null)
IMS.1b	onCountrySelected("ES")	showCities([Barcelona, Bilbao, Burgos, Cádiz, Gijón, León, Madrid, Oviedo, Santander], null)
IMS.1c	onCountrySelected("ES"), onCitySelected("Madrid") (Caso de éxito)	showNetworks([BiciMAD, Bici Metro])
IMS.1d	onCountrySelected("ES"), onCountrySelected("PT") (Cambio de país)	showCities([Lisboa], null) y showNetworks([Bici Lisboa])
IMS.1e	onCountrySelected("ES"), onCitySelected("Madrid"), onCountrySelected(null) («Todos»)	showNetworks con las 13 redes
IMS.1f	Ubicación en Santander (43.4623, -3.8099), onSelectCityClicked() (Ubicación por defecto)	showCitySelector([DE, AR, ES, PT], "ES", "Santander") y showNetworks([TUeBICI])

Reporte final

Resultado de la ejecución de las pruebas de la historia. Rellenar al terminar el Sprint.

Tabla 8. Resultado de las pruebas

Pruebas	Autor / ejecutor	Resultado
Unitarias CitySelector (UCS.1 – UCS.3)	Ángel González	Pendiente
Unitarias MainPresenter (UMP.2)	Diego Alonso	Pendiente
Integración MainPresenter (IMS.1)	Diego Alonso	Pendiente
Interfaz UI.1 – UI.2	Ángel González	Pendiente
Aceptación Pruebas 1 – 6	David González	Pendiente

Fallos detectados gracias a las pruebas: pendiente.

Anexo 1. Datos de prueba

Los datos con los que se lanza la aplicación en modo debug son los de res/raw/networks.json. Para esta historia se añaden tres redes (marcadas con *): una segunda red en Madrid, para tener una ciudad con dos redes, y dos redes en Portugal y Alemania, para tener cuatro países cuyo orden por nombre en español no coincide con el orden por código. Las redes nuevas solo tienen los datos básicos y no se abren sus estaciones en ninguna prueba. Bici Metro está a más de 5 km del punto de UNL.1b de la historia 532319, así que no cambia la red más cercana de esas pruebas.

Tabla 9. Redes de networks.json

id	Nombre	Ciudad	País	Coordenadas
bicing	Bicing	Barcelona	ES	41.3851, 2.1734
bicimad	BiciMAD	Madrid	ES	40.4168, -3.7038
bicimetro *	Bici Metro	Madrid	ES	40.3700, -3.7038
tuebici	TUeBICI	Santander	ES	43.4623, -3.8100
bicioeste	Bici Oeste	Oviedo	ES	43.3614, -5.8494
bicinorte	Bici Norte	Bilbao	ES	Sin longitud
bicicoord	Bici Coordenadas	Gijón	ES	Fuera de rango
bicianomala	Bici Anómala	León	ES	42.5987, -5.5671
bicitexto	Bici Texto	Burgos	ES	42.3439, -3.6969
bicisur	Bici Sur	Cádiz	ES	Sin coordenadas
bicilimites	Bici Límites	Ushuaia	AR	-90, 180
bicilisboa *	Bici Lisboa	Lisboa	PT	38.7223, -9.1393
biciberlin *	Bici Berlín	Berlín	DE	52.5200, 13.4050

Al pasar de 10 a 13 redes hay que actualizar las pruebas que comprueban el número de redes de la lista principal: DetectarCiudadSinUbicacionUITest y DetectarCiudadSinRedUITest.